

Experiment No: 3

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

1. Blockchain

A blockchain is a distributed and decentralized digital ledger used to store data or transactions in a secure and tamper-resistant manner. The data is stored in a sequence of blocks, where every block is connected to the previous block using its cryptographic hash.

Each block generally contains:

- Block index
- Timestamp
- Transaction data
- Proof/nonce used for mining
- Hash of the previous block

The connection between blocks makes unauthorized modification difficult because changing the data in one block changes its hash and affects all subsequent blocks.

The basic structure is:

Block 1 → Block 2 → Block 3 → Block 4

2. Cryptocurrency

A cryptocurrency is a digital form of currency that uses cryptographic techniques and blockchain technology to record and verify transactions.

In this experiment, a simple cryptocurrency called **HadCoin** is simulated using Python. Users can create transactions by specifying the sender, receiver, and amount.

Example:

Sender: Alice

Receiver: Bob

Amount: 10 HadCoin

The transaction is initially stored as a pending transaction and is later included in a block during mining.

3. Block

A block is a collection of transactions along with other information required to maintain the blockchain.

A block generally contains:

Index
Timestamp
Transactions
Proof
Previous Hash

The `previous_hash` field connects the current block to the previous block.

For example:

Block 1

↓

Block 2

↓

Block 3

If the information in Block 2 is modified, its hash changes and the connection with Block 3 becomes invalid.

4. Hashing

Hashing is the process of converting data into a fixed-length string called a hash.

The experiment uses the `hashlib` library and SHA-256 hashing.

A hash acts as a digital fingerprint of the data.

For example: `hashlib.sha256(data.encode()).hexdigest()`

Even a small change in the input data produces a different hash. Hashing therefore helps maintain the integrity of blockchain data.

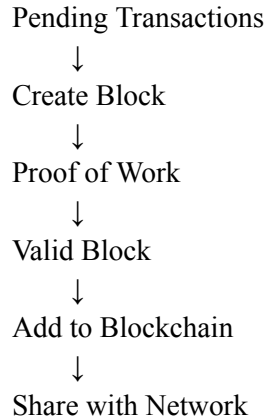
5. Mining

Mining is the process of creating a new block and adding it to the blockchain.

The mining process consists of:

1. Collecting pending transactions.
2. Creating a new block.
3. Adding a mining reward transaction.
4. Finding a valid proof using Proof-of-Work.
5. Adding the block to the blockchain.
6. Sharing the updated blockchain with other nodes.

The simplified mining process is:



6. Proof of Work

Proof of Work is a consensus mechanism in which a miner performs computational work to find a valid proof.

The program repeatedly tries different proof values until a hash satisfying the required condition is obtained.

This computational requirement makes it difficult to create blocks arbitrarily.

After a valid proof is found, the block can be added to the blockchain.

7. Peer-to-Peer Network

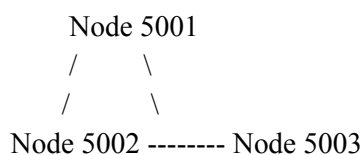
A Peer-to-Peer (P2P) network is a network in which nodes communicate directly with one another without depending on a single central server.

In this experiment, three blockchain nodes are created:

Node 5001
Node 5002
Node 5003

Each node maintains its own copy of the blockchain and can communicate with the other nodes using HTTP requests.

The network can be represented as:



This demonstrates how blockchain nodes communicate and synchronize their blockchain data.

8. Transactions

A transaction represents the transfer of cryptocurrency from one participant to another.

Each transaction contains:

- Sender
- Receiver
- Amount

Example:

Alice → Bob : 10 HadCoin

The transaction is first added to the list of pending transactions. A miner later collects these transactions and adds them to a block.

9. Mempool

A mempool, or memory pool, is a temporary collection of transactions that have been submitted to the blockchain network but have not yet been included in a block.

For example:

Mempool

Alice → Bob 10
John → Mike 5
Raj → Sara 20

During mining, the miner selects pending transactions from the mempool and includes them in a new block.

Once the transactions are successfully added to a block, they should be removed from the pending transaction list so that they are not included again in future blocks.

10. Mining Reward

In the implemented cryptocurrency, a reward transaction is added when a miner successfully mines a block.

The reward is given to the node/miner that performs the mining operation.

This demonstrates the basic concept of cryptocurrency mining rewards.

11. Consensus Mechanism

Consensus is the process by which different nodes in a distributed blockchain network agree on the valid version of the blockchain.

This experiment uses the **Longest Chain Rule**.

If a node discovers that another peer has a longer valid blockchain, it replaces its own shorter chain with the longer chain.

For example:

Node 5001 → 2 blocks

Node 5002 → 3 blocks

Node 5003 → 3 blocks

After consensus, Node 5001 can replace its 2-block chain with the valid 3-block chain.

This helps maintain consistency among the nodes.

12. Chain Replacement

The `replace_chain()` operation is used to synchronize a node with the longest valid blockchain available among its peers.

The process is:

1. The node contacts its registered peers.
2. It obtains their blockchain.
3. It checks the length of the received chains.
4. It validates the longer chains.
5. If a longer valid chain is found, the node replaces its own chain.

Thus, all nodes can eventually maintain a consistent blockchain.

Output:

POST http://127.0.0.1:5001/connect_node

Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "nodes": [
3     "http://127.0.0.1:5002",
4     "http://127.0.0.1:5003"
5   ]
6 }
```

Body Cookies Headers (5) Test Results 201 CREATED 12 ms 325 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
3   "total_nodes": [
4     "127.0.0.1:5002",
5     "127.0.0.1:5003"
6   ]
7 }
```

POST http://127.0.0.1:5001/connect_node

Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "nodes": [
3     "http://127.0.0.1:5001",
4     "http://127.0.0.1:5003"
5   ]
6 }
```

Body Cookies Headers (5) Test Results 201 CREATED 3 ms 342 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
3   "total_nodes": [
4     "127.0.0.1:5002",
5     "127.0.0.1:5003",
6     "127.0.0.1:5001"
7   ]
8 }
```

POST http://127.0.0.1:5001/connect_node



http://127.0.0.1:5001/connect_node

Save

POST

http://127.0.0.1:5001/connect_node

Send

Params

Authorization

Headers (8)

Body

Pre-request Script

Tests

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

JSON

Beautify

```
1
2  "nodes": [
3    "http://127.0.0.1:5001",
4    "http://127.0.0.1:5002"
5  ]
6
```

Body

Cookies

Headers (5)

Test Results



201 CREATED 3 ms 342 B

Save Response

Pretty

Raw

Preview

Visualize

JSON



```
1
2  "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
3  "total_nodes": [
4    "127.0.0.1:5002",
5    "127.0.0.1:5003",
6    "127.0.0.1:5001"
7  ]
8
```

POST http://127.0.0.1:5001/add_transaction



http://127.0.0.1:5001/add_transaction

Save

POST

http://127.0.0.1:5001/add_transaction

Send

Params

Authorization

Headers (8)

Body

Pre-request Script

Tests

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

JSON

Beautify

```
1
2  "sender": "Alice",
3  "receiver": "Bob",
4  "amount": 10
5
```

Body

Cookies

Headers (5)

Test Results



201 CREATED 4 ms 226 B

Save Response

Pretty

Raw

Preview

Visualize

JSON



```
1
2  "message": "This transaction will be added to Block 2"
3
```

GET http://127.0.0.1:5001/mine + ...



http://127.0.0.1:5001/mine_block

Save

GET

http://127.0.0.1:5001/mine_block

Send

Params

Authorization

Headers (8)

Body

Pre-request Script

Tests

Settings

Cookies

Type

No Auth

This request does not use any authorization. [Learn more about authorization](#)

Body

Cookies

Headers (5)

Test Results



200 OK 5 ms 510 B

Save Response

Pretty

Raw

Preview

Visualize

JSON



```
1  {
2    "index": 2,
3    "message": "Congratulations, you just mined a block!",
4    "previous_hash": "b03a910043992aaf088b9a6304c394d22b131a5b2c3899c0971bb4eb91f9fc5e",
5    "proof": 533,
6    "timestamp": "2026-08-27 09:06:29.957234",
7    "transactions": [
8      {
9        "amount": 10,
10       "receiver": "Bob",
11       "sender": "Alice"
12     },
13     {
14       "amount": 1,
15       "receiver": "Richard",
16       "sender": "341c9c8d81e1480dae2bca081e7e7845"
17     }
18   ]
19 }
```


GET http://127.0.0.1:5001/get_



http://127.0.0.1:5001/get_chain

Save

GET

http://127.0.0.1:5001/get_chain

Send

Params Authorization Headers (8) Body ● Pre-request Script Tests Settings

Cookies

Body Cookies Headers (5) Test Results

200 OK 3 ms 581 B Save Response

Pretty

Raw

Preview

Visualize

JSON



```
1  {
2    "chain": [
3      {
4        "index": 1,
5        "previous_hash": "0",
6        "proof": 1,
7        "timestamp": "2026-08-27 08:59:46.361705",
8        "transactions": []
9      },
10     {
11       "index": 2,
12       "previous_hash": "b03a910043992aaf088b9a6304c394d22b131a5b2c3899c0971bb4eb91f9fc5e",
13       "proof": 533,
14       "timestamp": "2026-08-27 09:06:29.957234",
15       "transactions": [
16         {
17           "amount": 10,
18           "receiver": "Bob",
19           "sender": "Alice"
```

```
12       "previous_hash": "b03a910043992aaf088b9a6304c394d22b131a5b2c3899c0971bb4eb91f9fc5e",
13       "proof": 533,
14       "timestamp": "2026-08-27 09:06:29.957234",
15       "transactions": [
16         {
17           "amount": 10,
18           "receiver": "Bob",
19           "sender": "Alice"
20         },
21         {
22           "amount": 1,
23           "receiver": "Richard",
24           "sender": "341c9c8d81e1480dae2bca081e7e7845"
25         }
26       ]
27     },
28   ],
29   "length": 2
30 }
```

```
GET http://127.0.0.1:5001/repl... + ...

http://127.0.0.1:5001/replace_chain Save

GET http://127.0.0.1:5001/replace_chain Send

Params Authorization Headers (8) Body ● Pre-request Script Tests Settings Cookies

Body Cookies Headers (5) Test Results 200 OK 54 ms 629 B Save Response

Pretty Raw Preview Visualize JSON

1 2 "actual_chain": [
3   {
4     "index": 1,
5     "previous_hash": "0",
6     "proof": 1,
7     "timestamp": "2026-08-27 08:59:46.361705",
8     "transactions": []
9   },
10  {
11    "index": 2,
12    "previous_hash": "b03a910043992aaf088b9a6304c394d22b131a5b2c3899c0971bb4eb91f9fc5e",
13    "proof": 533,
14    "timestamp": "2026-08-27 09:06:29.957234",
15    "transactions": [
16      {
17        "amount": 10,
18        "receiver": "Bob",
19        "sender": "Alice"
```

```
12    "previous_hash": "b03a910043992aaf088b9a6304c394d22b131a5b2c3899c0971bb4eb91f9fc5e",
13    "proof": 533,
14    "timestamp": "2026-08-27 09:06:29.957234",
15    "transactions": [
16      {
17        "amount": 10,
18        "receiver": "Bob",
19        "sender": "Alice"
20      },
21      {
22        "amount": 1,
23        "receiver": "Richard",
24        "sender": "341c9c8d81e1480dae2bca081e7e7845"
25      }
26    ]
27  },
28 ],
29 "message": "All good. The chain is the largest one."
30
```

Conclusion: We developed a cryptocurrency using Python and implemented mining in a simulated three-node blockchain network. Each node maintained its own ledger and synchronized with peers using the Longest Chain Rule to ensure consistency. Mining was performed through Proof-of-Work, securely adding transactions and rewarding miners. This demonstrated key blockchain concepts, including decentralized consensus, transaction validation, and network synchronization.